

App Development-2 Project Report

1. Student Details

Name: Soham Santosh Narayankhedkar

Roll Number: 24f2003116

Email: 24f2003116@ds.study.iitm.ac.in

About Me: I am an IIT Madras BS Degree in Data Science and Applications student, currently in diploma level with a strong interest in web application development and artificial intelligence. I enjoy building practical software systems that solve real-world problems and improve user experience. This project allowed me to apply backend development, database design, and advanced async communication to build a functional web-based placement portal.

2. Project Details

Project Title: Placement Portal Web Application (V2)

Problem Statement:

Many educational institutions require a system to efficiently manage placement drives, student applications, and company participation. Manual handling of this process is inefficient and prone to errors. The objective of this project is to build an advanced web-based placement portal that allows companies to post job drives, enforces admin approval checks, supports real-time caching for high performance, and relies on background workers for scheduling interview reminders and compiling monthly analytics reports.

Approach:

The system was developed as a modern web application using a Flask REST API backend serving a Vue.js 3 reactive client-side dashboard interface. It supports role-based access control for Admins, Companies, and Students. Students can register, upload resumes, filter active listings, and apply. Recruiter companies can create job postings, schedule interviews, and select candidates. Admins oversee verification and approval tasks. Caching is handled dynamically via Redis, while asynchronous tasks (CSV exports, PDF generation, and email reminders) are offloaded to Celery background workers.

3. AI / LLM Declaration

AI tools were used in a limited capacity during the development of this project.

Tools used:

- ChatGPT

AI assistance was mainly used for:

- Code suggestions
- Debugging support
- Improving documentation formatting

The estimated AI/LLM usage in this project is approximately around 30%, mainly for guidance and minor code improvements. The complete implementation logic, database schema design, debugging, and integration were performed manually. The following is the break-up:

- 1) Backend (Flask + DB) - 10% (For getting a brief overview of backend logic, session and route handling and fixing minor logical errors)
- 2) Frontend (Vue.js + Bootstrap CSS) - 5% (Getting to know how to use bootstrap and how template linking works)
- 3) Styling/UI - 5% (Adding appropriate buttons and colours by using bootstrap classes)
- 4) Auth + CRUD Logic - 10% (Adding of appropriate logic at frontend and backend for authentication and CRUD, and development of hashing technique for better security of passwords stored at the backend)

4. Technologies and Frameworks Used

Technology	Purpose
Flask	Core backend framework for REST APIs
Flask-Security-Too	Authentication and role-based authorization
Vue.js 3	Reactive frontend development
Axios	API communication between Vue and Flask
SQLite	Relational database
SQLAlchemy	ORM for database operations
Redis	API caching and Celery message broker
Celery	Background asynchronous jobs
Celery Beat	Scheduled background jobs
Bootstrap 5	Responsive UI styling
Jinja2	Entry point template rendering
ReportLab	PDF offer letter generation
SMTP (Gmail)	Email notifications
dbdiagram.io	Generating and visualizing ER database schemas

5. Database Schema Design

Tables:

- 1) User: Stores account credentials, roles (Admin/Company/Student), and SMTP settings.
- 2) Company: Stores corporate recruiter profile data, locations, and verification statuses.
- 3) Student: Stores student details, CGPA, graduation years, resume links, and blacklist flags.
- 4) PlacementDrive: Stores job titles, descriptions, deadlines, salary terms, and active states.
- 5) Application: Tracks registrations, scheduled interview timestamps, and meeting link addresses.
- 6) Placement: Records confirmed selections, salary pack terms, and PDF offer document links.

Relationships:

- One Company → Many Placement Drives
- One Placement Drive → Many Applications
- One Student → Many Applications
- One Student → One or More confirmed Placements

ER Diagram (Created using dbdiagram.io):

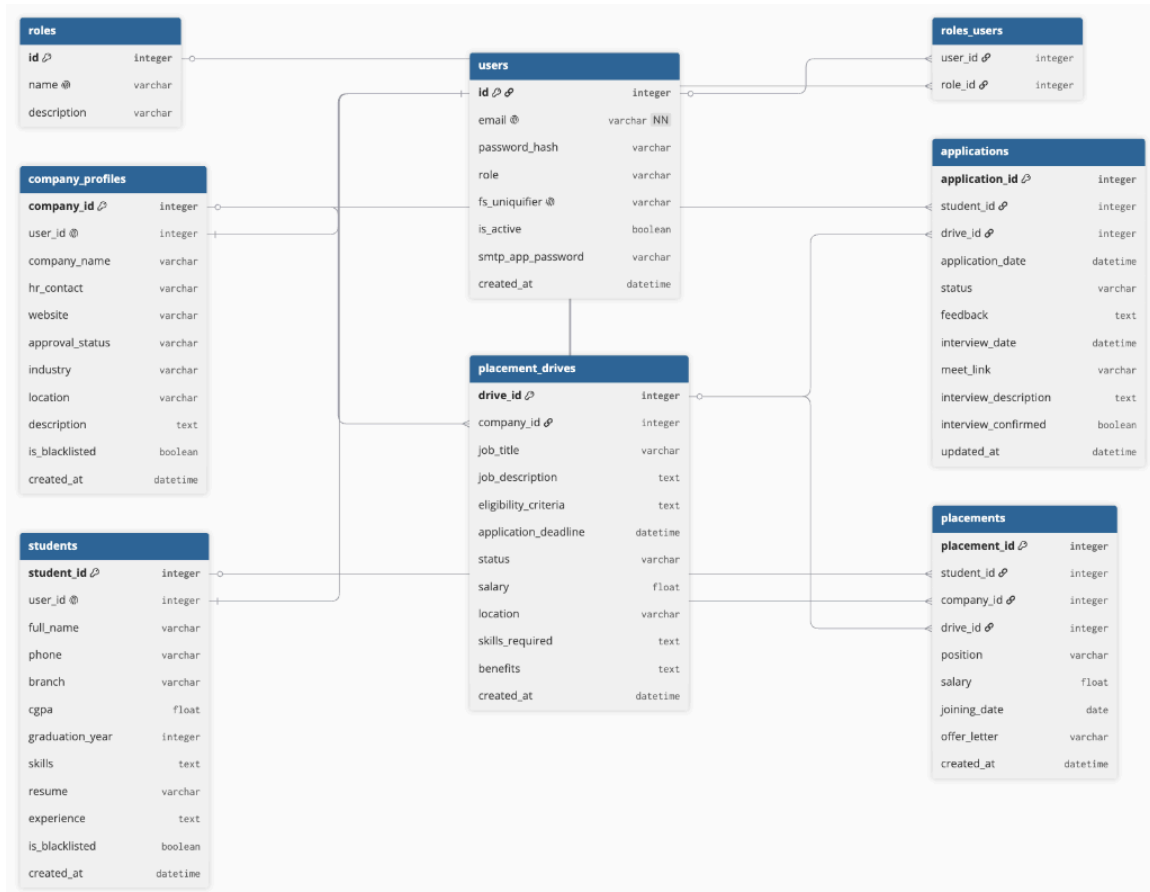


Figure: Entity-Relationship (ER) Diagram (Generated by dbdiagram.io)

6. API Resource Endpoints

The application uses HTTP routes implemented using Flask to handle various system operations such as authentication, job postings, and student applications. Examples of implemented endpoints include:

Endpoint	Method	Description
/login	GET, POST	Handles user authentication sessions.

/register	GET, POST	Registers student or company recruiter profiles.
/api/student/drives	GET	Fetches active drives (reads from Redis cache if available).
/api/student/drives/<int:id>/apply	POST	Submits student job application to a placement drive.
/api/company/drives	POST	Allows recruiters to publish new drives (requires admin approval).
/api/company/applications/<int:id>/status	POST	Updates candidate applications to Placed/Rejected states.
/api/admin/companies/<int:id>/approve	POST	Allows administrator to verify and approve recruiters.
/api/admin/reports	GET	Lists generated daily, weekly, and monthly PDF summaries.

7. Architecture and Features

The project follows a Flask-based MVC architecture. The backend logic and routing are implemented in app.py, which acts as the controller handling HTTP requests and database interactions. HTML templates located in the templates folder handle the frontend views (wrapped in Vue.js mounts), while static resources such as CSS, CSV exports, and resumes are stored in the static directory. The application uses a SQLite database to store user data, placement drives, and application records. This structure separates presentation, business logic, and data storage for easier maintenance.

The system supports three user roles: Admin, Company, and Student. Admins can manage users and monitor placements, companies can create placement drives and select candidates, and students can apply to drives and manage their profiles. Additional features include resume upload, applicant tracking, and placement status management.

8. Video Presentation

Drive Link:

https://drive.google.com/file/d/1ZCp9aQp8TslJH5CW_ALzPdWkBlorSq67/view?usp=sharing